Задание № 3. Прикладные дифференциальные уравнения (по материалам лекций).

©*Составитель: д.ф.-м.н., проф. Рябов П.Е.*

3.1.(Задача о равномерном спуске) Найти такую кривую, что тяжелая точка под действием силы тяжести равномерно опускается по этой кривой. Условие задачи предполагает, что точка движется по кривой, расположенной в некоторой вертикальной плоскости.

1) В ноутбуке (файл Jupyter Notebook) в разметке Markdown (LaTeX) привести подробное аналитическое решение (составление дифференциального уравнения, его решение);

2) Привести статическую картинку в формате png (pdf) решение задачи (код на Jupyter Notebook, который создает рисунок);

3) Используя инструментарий Jupyter Notebook привести анимацию данного решения.

Детализация задания для студентов:

Цель: Исследовать и визуализировать кривую, по которой материальная точка движется с постоянной вертикальной скоростью под действием силы тяжести.

Требования к выполнению:

1) **Аналитическое решение (Markdown/LaTeX):**
   - Ввести систему координат (ось x - горизонталь, ось y - вертикаль вверх)
   - Записать условие равномерного вертикального движения: $v_y = \text{const} < 0$
   - Составить дифференциальное уравнение из второго закона Ньютона
   - Исключить силу нормальной реакции
   - Проинтегрировать полученное уравнение
   - Найти явный вид кривой $y = f(x)$
2) **Статическая визуализация:**
   - Построить график найденной кривой для различных начальных условий
   - Отметить начальное положение точки
   - Добавить сетку, подписи осей, легенду
   - Сохранить рисунок в формате png

3) **Анимация решения:**
   - Использовать matplotlib.animation.FuncAnimation
   - Создать анимацию движения точки по кривой
   - Реализовать плавное движение с постоянной вертикальной скоростью
   - Добавить динамическое отображение параметров движения
   - Обеспечить возможность управления анимацией (пауза, воспроизведение)

Используемые библиотеки:

- matplotlib.pyplot для построения графиков
- numpy для математических вычислений
- matplotlib.animation.FuncAnimation для создания анимации
- matplotlib.widgets для интерактивных элементов управления

Критерии оценки:

- Корректность аналитического решения
- Качество и наглядность визуализации
- Работоспособность и плавность анимации
- Чистота и комментирование кода
- Полнота выполнения всех пунктов задания

Пример ожидаемого результата: Ноутбук, содержащий теоретическое решение, код построения кривой и интерактивную анимацию движения точки по найденной траектории.

3.2.(Задача о бильярде в прямоугольнике) Изучение движения материальной точки (бильярдного шара) в прямоугольной области с упругими отражениями от границ.

1) В ноутбуке (файл Jupyter Notebook) в разметке Markdown (LaTeX) привести подробное аналитическое решение (постановка задачи, метод развертки, анализ типов траекторий);
2) Привести статическую картинку в формате png (pdf) решение задачи (код на Jupyter Notebook, который создает рисунок);
3) Используя инструментарий Jupyter Notebook привести анимацию данного решения.

Детализация задания:

Цель: Исследовать и визуализировать движение бильярдного шара в прямоугольнике с упругими отражениями от границ, изучить периодические и эргодические траектории.

Требования к выполнению:

1) **Аналитическое решение (Markdown/LaTeX):**
   - Ввести систему координат и параметры прямоугольника (стороны a и b)
   - Записать уравнения движения и граничные условия для упругих отражений
   - Описать метод развертки и эквивалентное движение на торе
   - Проанализировать условия периодичности и эргодичности траекторий
   - Исследовать гамильтонову формулировку системы
2) **Статическая визуализация:**
   - Построить траекторию движения шара для различных начальных условий
   - Показать периодический и эргодический случаи
   - Визуализировать метод развертки на примере нескольких отражений
   - Добавить сетку, подписи осей, легенду
   - Сохранить рисунок в формате png
3) **Анимация решения:**
   - Использовать matplotlib.animation.FuncAnimation
   - Создать анимацию движения шара с упругими отражениями
   - Реализовать визуализацию метода развертки
   - Показать движение на торе для рациональных и иррациональных случаев
   - Добавить динамическое отображение параметров движения

Используемые библиотеки:

- matplotlib.pyplot для построения графиков
- numpy для математических вычислений
- matplotlib.animation.FuncAnimation для создания анимации
- matplotlib.widgets для интерактивных элементов управления

Критерии оценки:

- Корректность аналитического решения и описание метода развертки
- Качество и наглядность визуализации траекторий
- Работоспособность и плавность анимации

- Чистота и комментирование кода
- Полнота выполнения всех пунктов задания

Пример ожидаемого результата: Ноутбук, содержащий теоретическое решение, код построения траекторий и анимацию движения шара в прямоугольнике, включая демонстрацию периодических и непериодических случаев, а также визуализацию метода развертки.

3.3. (Задача о цепной линии) Исследовать форму равновесия однородной тяжелой нерастяжимой нити (цепи), подвешенной за два конца под действием силы тяжести.

1) В ноутбуке (файл Jupyter Notebook) в разметке Markdown (LaTeX) привести подробное аналитическое решение (вывод дифференциального уравнения, его решение);
2) Привести статическую картинку в формате png (pdf) решение задачи (код на Jupyter Notebook, который создает рисунок);
3) Используя инструментарий Jupyter Notebook привести анимацию данного решения.

Детализация задания для студентов:

Цель: Исследовать и визуализировать форму цепной линии - кривой равновесия тяжелой гибкой нити.

Требования к выполнению:

1) **Аналитическое решение (Markdown/LaTeX):**
   - Вывести дифференциальное уравнение из условий равновесия элемента цепи
   - Решить дифференциальное уравнение методом разделения переменных
   - Получить уравнение цепной линии в виде $y = a \cdot \mathrm{ch}(x/a)$
   - Проанализировать физический смысл параметра a
2) **Статическая визуализация:**
   - Построить график цепной линии для различных значений параметра a
   - Показать геометрические и физические свойства кривой
   - Добавить сетку, подписи осей, легенду
   - Сохранить рисунок в формате png
3) **Анимация решения:**
   - Использовать matplotlib.animation.FuncAnimation

- Создать анимацию изменения формы цепи при варьировании параметров
- Показать зависимость провисания от натяжения и плотности
- Визуализировать силы, действующие на элементы цепи

Используемые библиотеки:

- matplotlib.pyplot для построения графиков
- numpy для математических вычислений
- matplotlib.animation.FuncAnimation для создания анимации
- scipy.integrate для решения дифференциальных уравнений (опционально)

Критерии оценки:

- Корректность аналитического вывода дифференциального уравнения
- Качество визуализации цепной линии и её свойств
- Работоспособность и наглядность анимации
- Чистота и комментирование кода
- Полнота выполнения всех пунктов задания

Пример ожидаемого результата: Ноутбук, содержащий теоретическое решение, код построения цепной линии и анимацию, демонстрирующую изменение формы цепи при различных параметрах.

3.4. (Задача о таутохроне) Найти кривую, по которой время скатывания материальной точки под действием силы тяжести до нижней точки не зависит от начального положения (изохронная кривая).

1) В ноутбуке (файл Jupyter Notebook) в разметке Markdown (LaTeX) привести подробное аналитическое решение (постановка задачи, вывод интегрального уравнения, решение);
2) Привести статическую картинку в формате png (pdf) решение задачи (код на Jupyter Notebook, который создает рисунок);
3) Используя инструментарий Jupyter Notebook привести анимацию данного решения.

Детализация задания для студентов:

Цель: Исследовать и визуализировать таутохронную кривую (циклоиду), обеспечивающую постоянное время спуска.

Требования к выполнению:

1) **Аналитическое решение (Markdown/LaTeX):**
   - Составить интегральное уравнение для времени спуска
   - Решить интегральное уравнение Абеля
   - Получить параметрические уравнения циклоиды
   - Доказать независимость времени спуска от начальной точки
2) **Статическая визуализация:**
   - Построить график циклоиды и её геометрическое построение
   - Показать несколько траекторий с разных начальных высот
   - Визуализировать качение образующей окружности
   - Добавить сетку, подписи осей, легенду
3) **Анимация решения:**
   - Использовать matplotlib.animation.FuncAnimation
   - Создать анимацию одновременного скатывания шариков с разных высот
   - Показать геометрическое построение циклоиды
   - Визуализировать равенство времен спуска

Используемые библиотеки:

- matplotlib.pyplot для построения графиков
- numpy для математических вычислений
- matplotlib.animation.FuncAnimation для создания анимации
- sympy для аналитических вычислений (опционально)

Критерии оценки:

- Корректность решения интегрального уравнения Абеля
- Качество визуализации циклоиды и её свойств
- Наглядность демонстрации изохронности
- Работоспособность и плавность анимации
- Полнота выполнения всех пунктов задания

Пример ожидаемого результата: Ноутбук, содержащий теоретическое решение, код построения циклоиды и анимацию, наглядно демонстрирующую свойство таутохронности.

3.5. (Движение математического маятника) Исследовать движение математического маятника - точечной массы на невесомом нерастяжимом стержне в поле тяжести.

1) В ноутбуке (файл Jupyter Notebook) в разметке Markdown (LaTeX) привести подробное аналитическое решение (вывод уравнения движения, анализ решений);
2) Привести статическую картинку в формате png (pdf) решение задачи (код на Jupyter Notebook, который создает рисунок);
3) Используя инструментарий Jupyter Notebook привести анимацию данного решения.

Детализация задания для студентов:

Цель: Исследовать и визуализировать движение математического маятника, включая линейные и нелинейные режимы.

Требования к выполнению:

1) **Аналитическое решение (Markdown/LaTeX):**
   - Вывести уравнение движения из второго закона Ньютона
   - Проанализировать линеаризованное решение для малых углов
   - Исследовать нелинейные эффекты и точное решение через эллиптические интегралы
   - Построить фазовый портрет системы
2) **Статическая визуализация:**
   - Построить графики углового отклонения для различных начальных условий
   - Создать фазовый портрет с выделением различных режимов движения
   - Показать зависимость периода от амплитуды
   - Визуализировать сепаратрису и области колебаний/вращений
3) **Анимация решения:**
   - Использовать matplotlib.animation.FuncAnimation
   - Создать анимацию движения маятника в конфигурационном пространстве
   - Реализовать одновременную визуализацию на фазовой плоскости
   - Показать переход от гармонических к нелинейным колебаниям

Используемые библиотеки:

- matplotlib.pyplot для построения графиков
- numpy для математических вычислений

- matplotlib.animation.FuncAnimation для создания анимации
- scipy.integrate для численного решения уравнений
- scipy.special для эллиптических интегралов (опционально)

Критерии оценки:

- Корректность вывода и анализа уравнения движения
- Качество визуализации различных режимов движения маятника
- Наглядность фазового портрета и выделения особых траекторий
- Работоспособность и синхронизация анимаций
- Полнота исследования линейных и нелинейных эффектов

Пример ожидаемого результата: Ноутбук, содержащий теоретическое решение, код моделирования маятника и анимации, демонстрирующие различные режимы движения и фазовый портрет системы.